

Sets & Choosing the Right Collection

Unique values, membership tests, and collection intent

“Do we have this value?”



```
allowed_statuses = {  
    "active",  
    "inactive",  
    "retired",  
}  
  
status in allowed_statuses
```



Sets are for unique values
and fast membership
thinking.

Today's mission

Learn sets, then choose between
list, tuple, dictionary, and set
based on what the data means.

What is a set?

A set is a collection of unique values.

```
departments = {  
    "engineering",  
    "operations",  
    "finance",  
}
```

Key properties

- unique values only
- great for membership checks
- not used for position-based access
- order is not the point

Mental model

A set is like a checklist of allowed or observed values
—
not a numbered list.

BLOCK 20

Duplicates collapse automatically

A set keeps one copy of each unique value.

```
values = {"active", "active", "inactive"}
```



active

inactive

Only unique values remain.

```
record_ids = ["EQ-1001", "EQ-1002", "EQ-1001"]  
unique_ids = set(record_ids)
```

Useful for duplicate
detection.

Membership testing

Use in when you care whether a value belongs.

```
allowed_statuses = {  
    "active",  
    "inactive",  
    "retired",  
}  
  
if status in allowed_statuses:  
    print("Valid status")
```

Cleaner than repeated or comparisons

```
if (  
    status == "active"  
    or status == "inactive"  
    or status == "retired"  
):  
    print("Valid status")
```

This connects directly to validation rules.

BLOCK 20

Adding and removing values

Sets can change as rules change.

```
allowed_statuses.add("maintenance")
```

Add one new allowed value

```
allowed_statuses.remove("retired")
```

Remove a value from the group

Practical example

When a system starts accepting a new file
type,
add it to the allowed set.

Set operations

Compare groups of values.

Intersection

`matching = required & employee`

values in both

Difference

`missing = required - employee`

required but absent

Union

`all_skills = required | employee`

all unique values

```
required_skills = {"python", "sql", "git"}  
employee_skills = {"python", "excel", "sql"}
```

Sets answer useful questions about overlap and gaps.

Intersection: values in both

What requirements does the employee already meet?

```
required_skills = {"python", "sql", "git"}  
employee_skills = {"python", "excel", "sql"}  
  
matching_skills = required_skills & employee_skills
```

required

python
sql

employee

Intersection gives the overlap.

Difference: what is missing?

Required values not present.

```
missing_skills = required_skills - employee_skills  
  
print(missing_skills)
```

Output

```
{ "git" }
```

Why it matters

Difference is perfect for required-field checks, training gaps, missing permissions, or unavailable data.

Union: combine without duplicates

All unique values from both sets.

```
all_skills = required_skills | employee_skills  
print(all_skills)
```

One combined set

python

sql

git

excel

Union is useful when merging categories,
capabilities, tags, or observed values.

Choosing a collection

The best structure depends on intent.

List

ordered values
may change
duplicates matter

Tuple

fixed group
order matters
should not change

Dictionary

named fields
structured record
key-value data

Set

unique values
membership tests
set comparisons

Ask: “How will I use this data?”

List versus set

Same values, different purpose.

```
record_ids = [  
    "EQ-1001",  
    "EQ-1002",  
    "EQ-1001",  
]
```

```
unique_ids = set(record_ids)
```

List

Keeps order and
duplicates

Set

Keeps unique values only

Do not use a set when duplicates or order are meaningful.

Combining collection types

Real programs stack simple ideas together.

```
records = [  
  {  
    "record_id": "EQ-1001",  
    "status": "active",  
  },  
  {  
    "record_id": "EQ-1002",  
    "status": "inactive",  
  },  
]
```

Outer structure

List = many records

Inner structures

Dictionaries = named fields

This pattern becomes extremely important.

Pattern: list of dictionaries

A small dataset in plain Python.

```
records = [  
    {"record_id": "EQ-1001", "status": "active", "error_count": 0},  
    {"record_id": "EQ-1002", "status": "active", "error_count": 3},  
    {"record_id": "EQ-1003", "status": "inactive", "error_count": 0},  
    {"record_id": "EQ-1004", "status": "active", "error_count": 7},  
]
```

```
first_record = records[0]  
print(first_record["record_id"])
```

List index → dictionary key

Guided exercise 1: allowed statuses

Use a set to validate text values.

```
allowed_statuses = {  
    "active",  
    "inactive",  
    "retired",  
}  
  
status = "ACTIVE"  
normalized_status = status.strip().lower()  
  
is_valid = normalized_status in allowed_statuses
```

Student tasks

- test active
- test retired
- test maintenance
- test ACTIVE before and after normalization

Exercise time: 6 minutes

Guided exercise 2: detect duplicates

Compare list length with set length.

```
record_ids = [  
    "EQ-1001",  
    "EQ-1002",  
    "EQ-1003",  
    "EQ-1002",  
    "EQ-1004",  
]  
  
has_duplicates = len(record_ids) != len(set(record_ids))
```

Why it works

The list counts every entry.
The set counts each unique value
once.

If the counts differ,
at least one duplicate exists.

BLOCK 20

Guided exercise 3: required fields

Use set difference to find gaps.

```
required_fields = {  
    "record_id",  
    "status",  
    "file_name",  
    "record_count",  
}  
  
available_fields = {  
    "record_id",  
    "status",  
    "file_name",  
}  
  
missing_fields = required_fields - available_fields
```

Result

```
{ "record_count" }
```

This is validation logic
without a long chain of or
comparisons.

Exercise time: 6 minutes

BLOCK 20

Guided exercise 4: select the collection

Choose by intent, not habit.

Daily temperatures	List	Latitude + longitude	Tuple
Employee record	Dictionary	Allowed statuses	Set
Processing steps	List	Unique department names	Set
RGB color components	Tuple	Validation messages	List

Ask students to defend each choice.

BLOCK 20

Applied challenge: build a dataset

Create a list containing at least four dictionaries.

```
records = [  
    {"record_id": "EQ-1001", "status": "active", "error_count": 0},  
    {"record_id": "EQ-1002", "status": "active", "error_count": 3},  
    {"record_id": "EQ-1003", "status": "inactive", "error_count": 0},  
    {"record_id": "EQ-1004", "status": "active", "error_count": 7},  
]
```

Dataset structure

Outer list = collection of
records

Each dictionary = one
record

Challenge time: 12 minutes

Applied challenge tasks

Access, modify, and explain the dataset.

- Explain why the outer structure is a list.
- Explain why each record is a dictionary.
- Access the first record.
- Retrieve a field from the first record.
- Modify one field.
- Determine the number of records.

```
first_record = records[0]
print(first_record["record_id"])

records[1]["error_count"] = 0

print(len(records))
```

**Two-step access
pattern**

```
records[0]["record_id"]
```

Applied challenge extension

Use sets to summarize the dataset manually.

```
statuses = {  
    records[0]["status"],  
    records[1]["status"],  
    records[2]["status"],  
    records[3]["status"],  
}  
  
print(statuses)
```

Even without loops yet,
sets can identify unique
observed values.

Coming soon

Loops will replace repeated
manual access.

BLOCK 20

Common set mistakes

Sets are simple, but they are not lists.

Trying to index a set

```
allowed_statuses[0]
```

Use membership testing
instead.

Expecting duplicates

```
{"active", "active"}
```

Only one active remains.

Forgetting normalization

```
"CSV" in {"csv"}
```

Normalize first.

Choose a set only when uniqueness is the point.

BLOCK 20

Morning knowledge check

Collection vocabulary in plain language.

- 1 What does a set do with duplicate values?
- 2 When is a dictionary better than a list?
- 3 What does `required_fields - available_fields` mean?
- 4 Why is `records[0]["status"]` a two-step lookup?
- 5 Which collection would you use for allowed file extensions?

Predict → run → explain.

BLOCK 20

Wrap-up: the collection toolbox

You now have four ways to group data.

List

[78, 84, 91]
ordered items

Tuple

(30.4, -86.9)
fixed group

Dictionary

{"id": "EQ-1"}
named fields

Set

{"csv", "json"}
unique values

```
records = [  
    {"record_id": "EQ-1001", "status": "active"},  
    {"record_id": "EQ-1002", "status": "inactive"},  
]
```

Next: loops let us process every item automatically.